

1. Project Title:

Web Platform for Coding Practice and Performance Assessment (CodeArena)

2. Project Statement and Outcomes

Developers preparing for technical interviews and competitive programming need an accessible platform to practice algorithm and data-structure problems, run their code against validated test cases in real time, and track improvement over time. The hardest engineering problem in a system like this is not the UI — it is building a secure code execution environment: user-submitted code must run in isolation so it cannot touch the host system, must be constrained in CPU/memory so a bad submission (e.g. an infinite loop) cannot hang the server, and must return a deterministic, repeatable verdict every time.

This project builds a full-stack Java coding-practice platform — “CodeArena” — using Spring Boot on the backend and React.js on the frontend. It features a sandboxed code execution engine built on Docker-based subprocess isolation, a structured problem bank supporting multiple languages, configurable test cases, and fully automated evaluation. A rich React.js code editor powered by Monaco Editor (the same editor that powers VS Code) gives users a professional, in-browser coding experience.

The deliverable is a complete platform where users browse problems by difficulty and topic, write code in Java, Python, C++ or JavaScript inside the Monaco editor, submit solutions, and instantly receive a verdict with per-test-case pass/fail detail, execution time, and memory usage. A personal dashboard tracks problems solved, success rate, language proficiency, and a GitHub-style streak calendar. A platform-wide leaderboard ranks users by problems solved, accuracy and speed, and an admin panel lets staff create problems, manage test cases, tag difficulty, and review platform analytics.

Teaching note: This project is an excellent vehicle for teaching students how production-grade systems isolate untrusted code, manage asynchronous workloads, and design APIs that have to be both fast and safe. Use it to connect classroom concepts (OS process control, containers, REST design, concurrency) to a real, demo-able product.

3. Modules to be Implemented

- User Auth, Problem Bank Schema & Sandboxed Code Execution Engine
- Submission Evaluation, Test Case Management & Leaderboard System
- React.js Monaco Editor, Problem Browser & Progress Dashboard
- System Integration, Testing & Project Finalization

4. Milestone-wise Module Implementation and High-Level Requirements

Each week below lists the objectives, the concrete features to build, the core computer-science / engineering concepts you should explain to students as you go, and the deliverables that mark the week complete.

Milestone 1: User Auth, Problem Bank Schema & Sandboxed Code Execution Engine

Objectives

- Stand up the data model that every other module depends on.

- Secure the platform with authentication and authorization before any business logic is built.
- Build and prove out the riskiest piece of the system first: running untrusted code safely.

Features to Implement

Database & Schema

- Design a PostgreSQL schema for users, problems, test_cases, submissions, submission_results (one row per test case), and leaderboard_entries.
- Model difficulty and submission status as enum-typed fields (e.g. EASY/MEDIUM/HARD, PENDING/RUNNING/ACCEPTED...).
- Map all tables to JPA entities and manage schema evolution with Flyway migrations instead of hand-edited SQL.

Authentication & Authorization

- Implement Spring Security 6 with JWT for registration and login.
- Add an ADMIN role with elevated permissions for problem management endpoints.
- Enforce submission rate limiting — a maximum of 5 submissions per problem per minute per user — to stop abuse and accidental floods.

Sandboxed Code Execution Engine

- Run user code as a Docker subprocess, one container per language: openjdk:17 for Java, python:3.11 for Python, gcc:latest for C++.
- Apply hard resource limits to every container: 512 MB memory and a 5-second CPU timeout.
- Launch and manage the container lifecycle from Java using ProcessBuilder, capturing stdout, stderr and exit codes.

Compilation Handling

- Compile the submitted code inside the container first.
- If compilation fails (syntax/type errors), return a distinct “compilation failure” response before any test case is run, so the user gets a fast, clear signal.

Testing the Engine

- Write JUnit 5 tests for: valid code that produces correct output, an infinite loop (timeout handling), a memory-exceeding program (OOM handling), and code that attempts a malicious system call (proves container isolation actually holds).

Concepts to Explain to Students

- **Process isolation vs. containers:** explain why running eval() or exec() directly on the host is dangerous, and how a container gives a separate filesystem, process namespace and resource cgroup.
- **Resource limiting:** walk through how CPU timeouts and memory caps are enforced at the OS/container level, not just in application code.
- **JWT authentication:** show the difference between a stateless token and a server-side session, and why stateless auth scales better for an API-driven app.
- **Database migrations:** contrast hand-run SQL scripts with Flyway-managed, version-controlled migrations.

Deliverables

- Working PostgreSQL schema with Flyway migrations checked into version control.
- JWT-based registration/login with role-based access control and rate limiting.

- A sandboxed execution engine that can compile and run code in Java, Python and C++ inside resource-limited Docker containers.
 - A JUnit test suite proving timeout, OOM, and isolation behaviour.
-

Milestone 2: Submission Evaluation, Test Case Management & Leaderboard System

Objectives

- Turn raw container output into a graded verdict.
- Give admins full control over problem content and difficulty.
- Reward consistent practice with rankings, hints and editorials.

Features to Implement

Test Case Evaluation

- Run the compiled code against every test case, both hidden and visible to the user.
- Compare actual stdout against expected output with whitespace normalization (trailing spaces/newlines should not fail a correct solution).
- Record pass/fail, execution time and memory used for every individual test case.

Problem Management API (Admin)

- Allow admins to create problems with title, Markdown description, difficulty, topic tags, and starter code per language.
- Support configurable test cases per problem: input, expected output, and an optional time-limit override for that case.

Submission Lifecycle

- Model the full state machine: pending → running → accepted / wrong-answer / time-limit-exceeded / memory-limit-exceeded / runtime-error.
- Persist full test-case-level detail against every submission for later review.

Leaderboard System

- Rank users by total problems solved (weighted by difficulty), acceptance rate, and fastest accepted submission time.
- Provide global, weekly, and per-language leaderboard views.

Hints & Editorials

- Let problems include step-by-step hints revealed on demand.
- Unlock a full editorial solution only after a configurable number of failed attempts, to encourage independent problem-solving first.

Concepts to Explain to Students

- **Verdict pipeline:** explain how a single submission fans out into N test-case executions and folds back into one overall verdict.
- **State machines:** use the submission lifecycle (pending → running → accepted/...) as a concrete example of modelling state transitions in a relational schema.
- **Ranking algorithms:** discuss why a naive “most solved” leaderboard is gameable, and how weighting by difficulty and speed produces a fairer ranking.

Deliverables

- Full per-test-case evaluation pipeline with output normalization.
- Admin APIs to create and manage problems and their test cases.
- A persisted submission lifecycle with detailed history.
- Global, weekly and language-specific leaderboards.
- Hint system and gated editorial solutions.

Milestone 3: React.js Monaco Editor, Problem Browser & Progress Dashboard

Objectives

- Give users a professional, IDE-like coding experience in the browser.
- Make problems easy to discover and filter.
- Visualize a user's progress to keep them motivated.

Features to Implement

Monaco Editor Integration

- Embed Monaco Editor (the engine behind VS Code) in React.js.
- Support language-specific syntax highlighting, auto-indentation and keyboard shortcuts for Java, Python, C++ and JavaScript.

Problem-Solving Interface

- Build a split-pane layout: rendered Markdown problem description on the left, Monaco editor on the right.
- Add tabs for sample test case input/output, plus Run (sample cases only) and Submit buttons.

Real-Time Submission Feedback

- Display per-test-case results as a verdict table (pass/fail with execution time).
- Show an overall verdict banner and surface compiler error messages with line-number highlights.

Problem Browser

- Add difficulty filters (Easy/Medium/Hard), topic-tag filters (Arrays, Trees, DP...), and title search.
- Show a solved/unsolved indicator and each problem's acceptance rate.

Progress Dashboard

- Show a problems-solved counter broken down by difficulty.
- Add a GitHub-style submission heatmap calendar, a language-usage breakdown, a recent-submissions table, and the user's current leaderboard rank.

Concepts to Explain to Students

- **Component-driven UI:** show how the problem-solving screen decomposes into editor, description, verdict-table and tab components.
- **Polling vs. WebSockets:** discuss how the frontend should fetch a submission's evolving status (pending → running → verdict) without freezing the UI.
- **Data visualization basics:** use the heatmap calendar and language breakdown to introduce simple client-side aggregation and charting.

Deliverables

- A working Monaco-based code editor supporting four languages.

- A complete problem-solving screen with live verdicts and error highlighting.
 - A filterable, searchable problem browser.
 - A personal progress dashboard with heatmap, language stats and rank.
-

Milestone 4: System Integration, Testing & Project Finalization

Objectives

- Prove the whole pipeline works end to end under realistic conditions.
- Polish the experience based on real feedback.
- Document and present the system as a complete, professional deliverable.

Features to Implement

End-to-End Integration

- Test the full chain: problem bank → code submission → Docker sandbox execution → test case evaluation → React Monaco editor → leaderboard.
- Resolve any cross-module compatibility issues and edge-case failures found along the way.

Performance & Cross-Browser Testing

- Load- and stress-test the Spring Boot REST APIs with Apache JMeter or k6.
- Validate the UI on Chrome, Firefox and Edge, and address any performance bottlenecks found during peer/mentor review.

UX Refinement

- Improve interface clarity, add contextual help and input validation.
- Ensure full responsiveness on mobile and tablet, and polish transitions and error states across all React components.

Documentation

- Document system architecture and entity-relationship diagrams.
- Generate an API reference with Swagger/OpenAPI via springdoc-openapi.
- Write a deployment guide using Docker Compose, list the Flyway migration scripts, and record known limitations.

Final Presentation

- Prepare a final PPT covering problem statement, proposed solution, architecture, key features, technology stack, results, challenges, and future scope.
- Rehearse a full end-to-end demo so the delivery is smooth and confident.

Concepts to Explain to Students

- **Load testing:** explain the difference between functional tests (does it work) and load tests (does it still work under concurrent users).
- **API documentation:** show how OpenAPI/Swagger specs double as both documentation and a contract for the frontend team.

Deliverables

- A fully integrated, end-to-end tested platform.
- Load/stress test results and cross-browser validation sign-off.

- A polished, responsive UI with refined error and loading states.
- Complete documentation: architecture, ER diagrams, API reference, deployment guide.
- A rehearsed final presentation and live demo.

5. System Flow

The flow chart supplied with this brief traces a single submission end to end: the user submits code from the Monaco editor → the API performs a JWT auth and rate-limit check → the submission is enqueued asynchronously (Spring Async) → it executes inside the Docker sandbox → on successful execution, output is compared against the test cases → the verdict is persisted and the leaderboard updated → the result is returned to the React UI. Walking students through this diagram before coding is a good way to set shared vocabulary for the whole project.

6. Sample Dashboard Reference

The supplied “CodeArena” dashboard mock-up shows the kind of admin/overview screen students should aim to reproduce in Weeks 5–6: headline stats (active users, submissions today, problems, live contests), a submissions-by-language chart, a live-contests panel, and a recent-submissions table with per-row verdicts. Use it as the visual target for the Problem Browser and Progress Dashboard work.

7. Recommended Technology Stack

Layer	Technology	Purpose
Backend Framework	Spring Boot 3 (Java 17)	REST APIs, dependency injection, application structure
Security	Spring Security 6 + JWT	Authentication, role-based authorization
Database	PostgreSQL + Flyway	Persistent storage and versioned schema migrations
ORM	Spring Data JPA / Hibernate	Entity mapping and query generation
Sandboxing	Docker (openjdk:17, python:3.11, gcc:latest)	Isolated, resource-limited code execution per language
Frontend Framework	React.js	Component-based single-page application
Code Editor	Monaco Editor	In-browser, VS Code-grade editing experience
API Docs	springdoc-openapi (Swagger)	Auto-generated, browsable API reference
Testing	JUnit 5, Apache JMeter / k6	Unit testing and load/stress testing

Layer	Technology	Purpose
Deployment	Docker Compose	Reproducible multi-container local/staging deployment

8. Core Database Entities

Entity	Key Fields	Notes
users	id, username, password_hash, role	role is enum: USER / ADMIN
problems	id, title, description_md, difficulty, tags	difficulty is enum: EASY / MEDIUM / HARD
test_cases	id, problem_id, input, expected_output, time_limit_override	Hidden vs. visible flag controls what the user sees
submissions	id, user_id, problem_id, language, status, submitted_at	status is the lifecycle enum (pending...accepted...)
submission_results	id, submission_id, test_case_id, passed, exec_time_ms, memory_kb	One row per test case per submission
leaderboard_entries	id, user_id, problems_solved, accuracy, rank	Recomputed periodically or on submission

9. Learning Outcomes for Students

- Designing a relational schema for a stateful, multi-table workflow and managing it with versioned migrations.
- Implementing stateless authentication and role-based authorization with Spring Security and JWT.
- Safely executing untrusted, third-party code using OS-level process isolation and container resource limits.
- Modelling and persisting a multi-step lifecycle (a submission's state machine) in a relational database.
- Building a polished, component-based React UI on top of a professional code-editing library (Monaco).
- Writing meaningful automated tests for security-sensitive and timing-sensitive code, not just “happy path” logic.
- Load testing and tuning a REST API, and documenting it for other engineers via OpenAPI/Swagger.